



CLAUDE CODE DEVELOPER ENABLEMENT PROGRAM

Day 1 — Foundations & the Agentic Mindset

From single-turn prompting to agentic delegation — the foundations every engineer needs before touching the tooling.

WAVE

[Wave / Location]

DATE

[Session Date]

DURATION

09:00 – 17:30



Day 1 Agenda

Foundations & the Agentic Mindset

MORNING

Arrival	Environment Spot-Check (TA-supported)
09:00	Pre-Training Survey & Expectations Capture
09:15	Wells Fargo Champion Spotlight
09:30	The Agentic Mindset Shift + Lab 1: WM-101
11:45	Break
12:00	Data Governance & Access Path

AFTERNOON

12:30	Lunch
13:30	Lab 2 · Backlog Audit + Your First Team Skill
15:15	Break
15:30	Lab 3 · WM-110: Drift Alert False Positives
16:30	Lab 4 · WM-106: Reconciliation Exception Triage Agent
17:30	Recap & Preview of Day 2

DAY 1
SCALE: INDIVIDUAL

Foundations & the Agentic Mindset Shift

Frame a task, delegate it to Claude Code, verify the result — then a workflow ladder of Skills, plan mode, and a first agent this afternoon.

DAY 2
SCALE: TEAM

Advanced Agentic Orchestration & CI/CD

The same shift, orchestrated across a team.

DAY 3
SCALE: FULL LIFECYCLE

Live Build: Vertical Modernization

The same shift, applied to a whole legacy module.

Pre-Training Survey & Expectations Capture

LIVE · MENTI

Join the live survey now
This slide is a placeholder — the real-time Menti survey will be projected here during delivery.



Go to `menti.com` · Enter code `XXXX XXXX`
Suggested questions: see `survey-questions.md`



09:15 – 09:30

Wells Fargo Champion Spotlight

A peer voice, not a vendor demo — a pilot user shares the workflow they changed, in their own words.

SPEAKER PLACEHOLDER
Name · Role · Team
photo/bio pending nomination

WHAT TO CAPTURE LIVE

- The workflow they changed — before and after.
- What it replaced, in their own words.
- What it bought them — time, confidence, or quality.



09:30 - 11:00 · THE AGENTIC MINDSET SHIFT

From Autocomplete to Agentic Delegation

Autocomplete (Claude or Copilot)
Completes lines and single files. You stay in control of the architecture — the tool fills in details.

Agentic Coding
Multi-file refactorings, multi-repo work. You delegate an outcome and review the result — the agent owns the approach.

TRADITIONAL DEV





09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

From Autocomplete to Agentic Delegation

Autocomplete (Claude or Copilot)
Completes lines and single files. You stay in control of the architecture — the tool fills in details.

Agentic Coding
Multi-file refactorings, multi-repo work. You delegate an outcome and review the result — the agent owns the approach.





09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

After This 90-Minute Session, You Will Be Able To...



09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

After This 90-Minute Session, You Will Be Able To...

- Describe the Read → Decide → Act → Observe loop, and name a task in your day where an agent adds value — and one where it doesn't.



09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

After This 90-Minute Session, You Will Be Able To...

- Describe the Read → Decide → Act → Observe loop, and name a task in your day where an agent adds value — and one where it doesn't.
- Package a task frame into a CLAUDE.md rule or a Skill so it runs the same loop without you rewriting it each time.



09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

After This 90-Minute Session, You Will Be Able To...

- Describe the Read → Decide → Act → Observe loop, and name a task in your day where an agent adds value — and one where it doesn't.
- Package a task frame into a CLAUDE.md rule or a Skill so it runs the same loop without you rewriting it each time.
- Ship one small, verified change, backed by test evidence — not just a claim that it works.



09:30 - 11:00 · THE AGENTIC MINDSET SHIFT

Meet the Loop: Read → Decide → Act → Observe





09:30 - 11:00 · THE AGENTIC MINDSET SHIFT

Meet the Loop: Read → Decide → Act → Observe



Good fit for the loop

A multi-file rename plus a regression sweep — many small Read→Decide→Act→Observe cycles, each cheap to verify, too tedious to want to do by hand.



09:30 - 11:00 · THE AGENTIC MINDSET SHIFT

Meet the Loop: Read → Decide → Act → Observe



Good fit for the loop

A multi-file rename plus a regression sweep — many small Read→Decide→Act→Observe cycles, each cheap to verify, too tedious to want to do by hand.

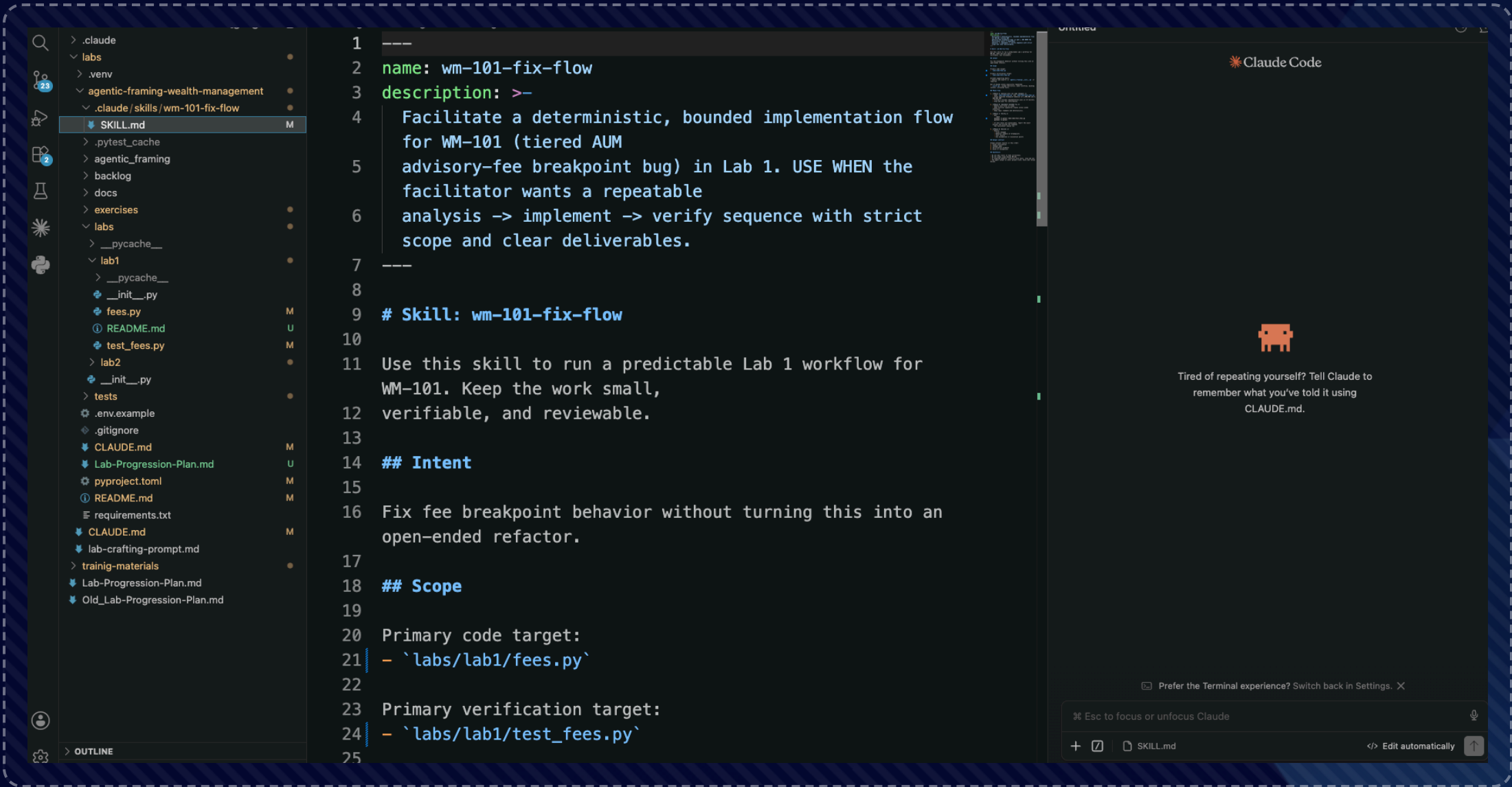
Poor fit for the loop

A one-line config toggle you could type yourself in ten seconds — framing and verifying it costs more than just making the edit.



09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

READ STAGE



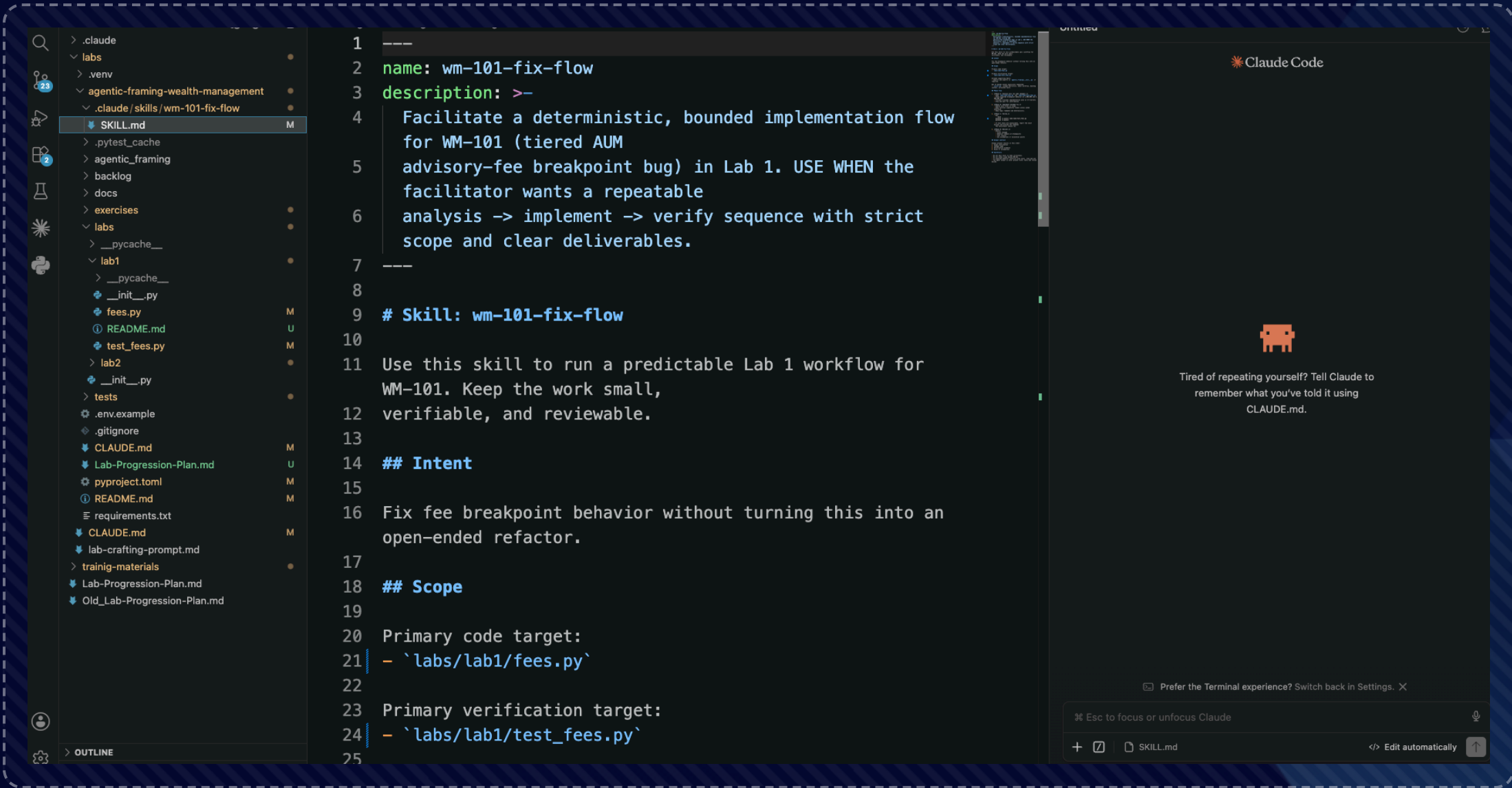
CLAUDE.md
Always-loaded project rules — the standing brief Claude Code reads before anything else.

.claude/skills/
Reusable, on-demand workflows — named skills like wm-101-fix-flow that package a team standard.



09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

READ STAGE



CLAUDE.md

Always-loaded project rules — the standing brief Claude Code reads before anything else.

.claude/skills/

Reusable, on-demand workflows — named skills like wm-101-fix-flow that package a team standard.

Copilot parallel: `.github/copilot-instructions.md` (repo-wide custom instructions) is Copilot's CLAUDE.md. Copilot Chat's **Prompt Files** (`.github/prompts/*.prompt.md`) are its closest analog to a Skill — reusable, named templates, though not a bounded, tool-using workflow the way a Skill is.



09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

READ STAGE - CLAUDE.md & Skills

Package the frame into one of these, and the loop runs again without you re-deciding it from scratch each time.

CLAUDE.md

Always-loaded project rules — the standing brief Claude Code reads before anything else.

.claude/skills/SKILL.md

A named, on-demand workflow — parameterized by the frame you hand it, not by the ticket.
scoped-bugfix-flow is the one Lab 1 uses.

WHERE PERSONAL BECOMES TEAM

```
CLAUDE.md x
labs > agentic-framing-wealth-management > CLAUDE.md > # Claude Code session guide - Day 1-2: the agentic_framing progression
1 # Claude Code session guide – Day 1-2: the agentic_framing progression
2 now. Read the
3 learner as a learner – coach the mindset, don't just hand over finished code.
4
5 The one big idea: this course telescopes
6
7 There are two nested agentic loops, and the same discipline shows up at every
8 zoom level:
9
10 1. You + Claude Code – this outer loop is governed by me ( CLAUDE.md ) and
11 the reusable
12 workflows in .claude/skills/. The engineer prompts Claude Code.
13 2. The app we're building ( agentic_framing ) – its "agent" is a static
14 prompt baked
15 into the code: converter.py 's SYSTEM_PROMPT . Later labs grow this inner
16 agent its own
17 SKILL.md and tools.
18 3. The task frame the app produces – the five ingredients below.
```

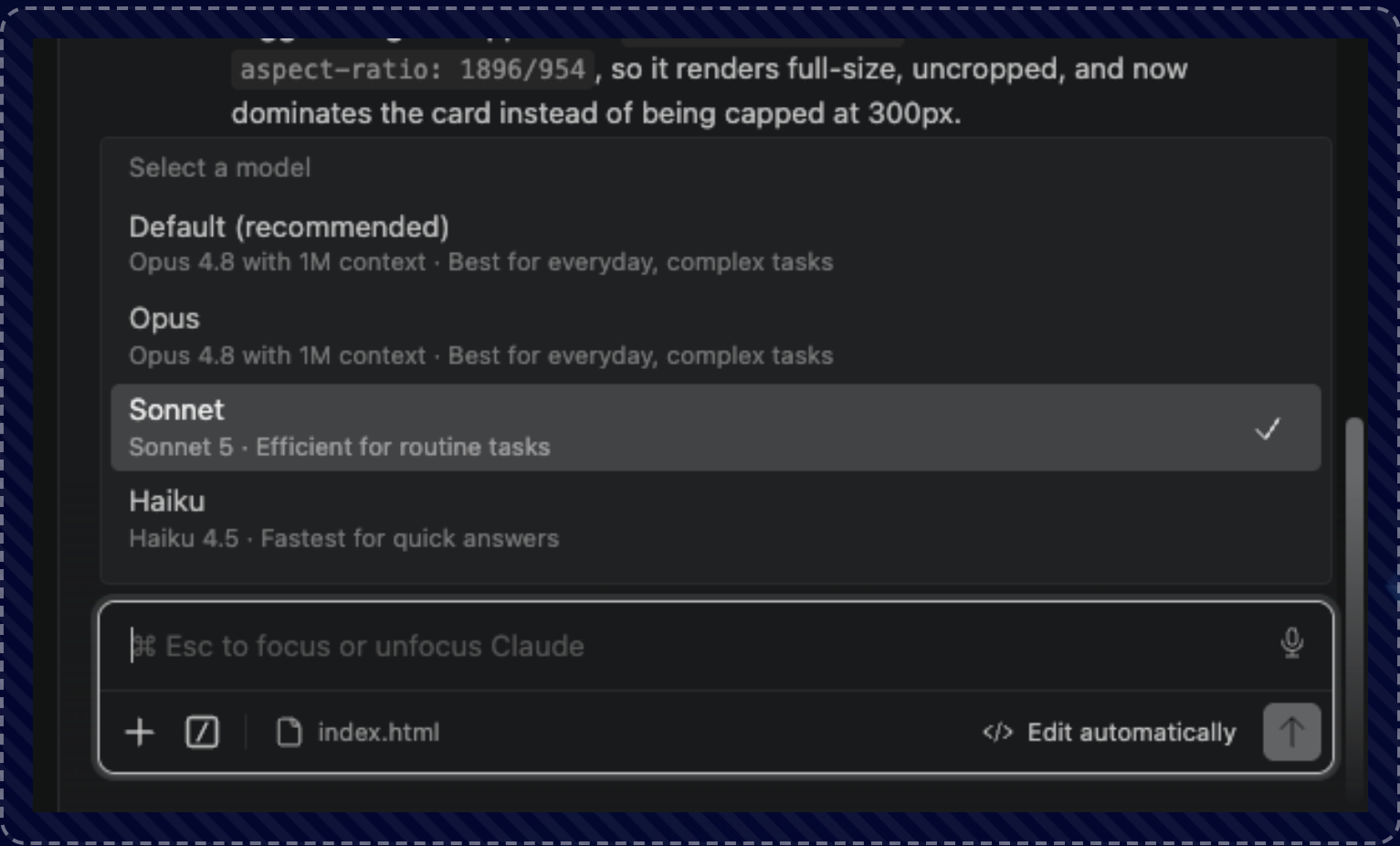
```
SKILL.md x
labs > agentic-framing-wealth-management > .claude > skills > scoped-bugfix-flow > SKILL.md > ...
1 The name of the skill.
2 name: scoped-bugfix-flow
3 description: >-
4 Run a deterministic, bounded bug-fix flow (analyze -> implement -> verify ->
5 deliver)
6 driven by a five-ingredient task frame (Outcome/Scope/Verification/Deliverable/
7 Guardrails).
8 USE WHEN the caller supplies a task frame for a scoped bug fix and wants a
9 repeatable,
10 reviewable sequence instead of an open-ended edit.
11 ---
12 Skill: scoped-bugfix-flow
13 Use this skill to run a predictable bounded bug-fix workflow for any ticket
14 that comes
15 with a five-ingredient task frame. Keep the work small, verifiable, and
16 reviewable. This
17 skill has no built-in knowledge of any specific ticket, file, or bug – every run
18 is
19 parameterized by the frame supplied in the invocation.
```

Copilot parallel: `.github/copilot-instructions.md` (repo-wide custom instructions) is Copilot's CLAUDE.md. Copilot Chat's **Prompt Files** (`.github/prompts/*.prompt.md`) are its closest analog to a Skill — reusable, named templates, though not a bounded, tool-using workflow the way a Skill is.



09:30 - 11:00 · THE AGENTIC MINDSET SHIFT

DECIDE STAGE



BEFORE YOU START

- Type /switch to bring up the options and select "Switch model".
- Fast tier for routine, formulaic steps; a stronger tier when framing or reasoning is the bottleneck.
- Don't pick out of habit — the right tier depends on the task. You just saw the benchmark.
- **Same pattern as the frame:** you're picking a tier by hand right now — Day 2's Lab 5 shows an orchestrator making this same call automatically, per subtask.



Decide Stage

OPUS 4.7

Precision Interpreter

Takes instructions literally and executes precisely on what you specify. Where earlier Opus versions hedged on ambiguity, 4.7 resolves it through action. Maintains coherence across very long outputs and sustains performance on complex multi-step tasks.

- High-stakes analysis
- Long-form reasoning
- Edge cases

SONNET 4.6 · NEW

Disciplined & Honest

Near-Opus reasoning at Sonnet speed. Significantly improved honesty: flags uncertainty rather than producing overconfident output. Reduced laziness on long tasks. The new default recommendation.

- New deployments
- Agentic coding
- Honesty-critical

HAIKU 4.5

Formulaic Executor

Follows instructions closely and consistently. Near-frontier intelligence for well-specified tasks at 4–5× Sonnet's speed. Best when you need predictable, high-throughput behavior, and the task is clearly defined.

- Classification
- Structured extraction
- Routing layers





09:30 - 11:00 · THE AGENTIC MINDSET SHIFT

Decide Stage

PICK YOUR MODE

Modes

Manual

Claude will ask for approval before making each edit

Edit automatically

Claude will edit your selected text or the whole file

Plan

Claude will explore the code and present a plan before editing

 + `tab` to switch

 Edit automatically

TWO MODES TO KNOW TODAY

- Plan mode** — Claude proposes a plan first; nothing executes until you approve it. Best when acceptance criteria or test shape is still unclear.
- Default (act) mode** — bounded edits run directly, with checkpoints on sensitive actions. Best once the target is locked.
- More automated modes exist for CI and governed pipelines — that's a Day 2 topic.



09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

ACT Stage

The one device you'll use all day — first by hand, then designed into something that runs without you.

THE FIVE-INGREDIENT FRAME





09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

ACT Stage

The one device you'll use all day — first by hand, then designed into something that runs without you.

THE FIVE-INGREDIENT FRAME

Outcome

The end state, not the keystrokes.



09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

ACT Stage

The one device you'll use all day — first by hand, then designed into something that runs without you.

THE FIVE-INGREDIENT FRAME

Outcome
The end state, not the keystrokes.

Scope
The concrete target and boundaries.



09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

ACT Stage

The one device you'll use all day — first by hand, then designed into something that runs without you.

THE FIVE-INGREDIENT FRAME

Scope
The concrete target and boundaries.

Outcome
The end state, not the keystrokes.

Verification
How "done" is checked — run it, don't eyeball it.



09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

ACT Stage

The one device you'll use all day — first by hand, then designed into something that runs without you.

THE FIVE-INGREDIENT FRAME

Scope
The concrete target and boundaries.

Outcome
The end state, not the keystrokes.

Deliverable
The reviewable artifact handed back.

Verification
How "done" is checked — run it, don't eyeball it.

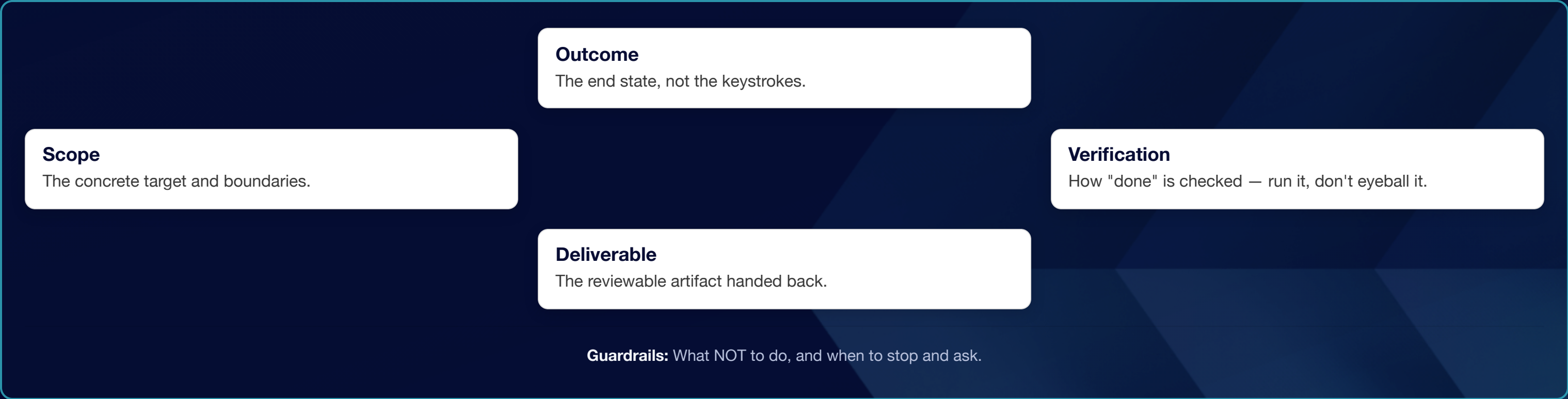


09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

ACT Stage

The one device you'll use all day — first by hand, then designed into something that runs without you.

THE FIVE-INGREDIENT FRAME



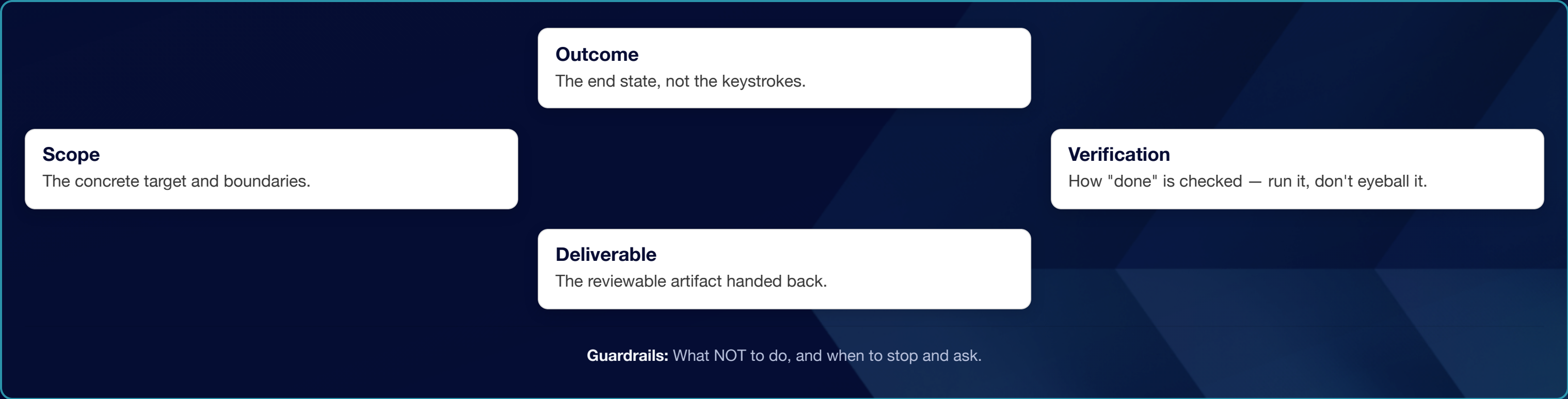


09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

ACT Stage

The one device you'll use all day — first by hand, then designed into something that runs without you.

THE FIVE-INGREDIENT FRAME



Write it, run it once
You write the frame by hand and drive Claude Code (or Copilot Chat) through one task, start to finish. **This is Lab 1.**

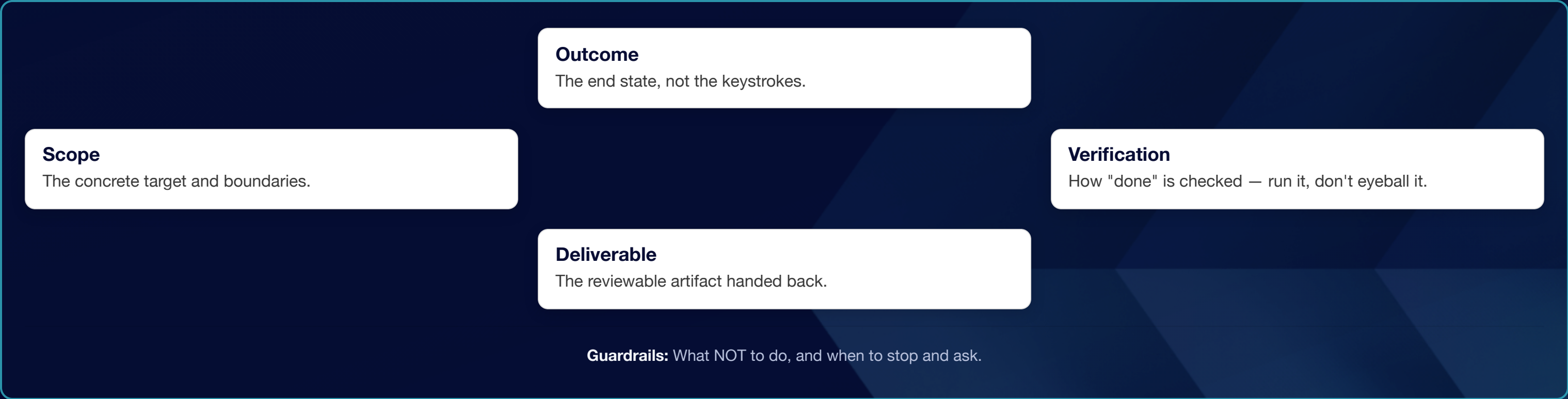


09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

ACT Stage

The one device you'll use all day — first by hand, then designed into something that runs without you.

THE FIVE-INGREDIENT FRAME



Write it, run it once

You write the frame by hand and drive Claude Code (or Copilot Chat) through one task, start to finish. **This is Lab 1.**

Design it in, it runs without you

You bake the same frame into a Skill you invoke on demand, or an agent that runs it autonomously. **This is Lab 2 → Lab 4.**



09:30 - 11:00 · THE AGENTIC MINDSET SHIFT

The Frame in Practice: Poor, Better, Best

Same task — reviewing open reconciliation exceptions — framed four ways.



09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

The Frame in Practice: Poor, Better, Best

Same task — reviewing open reconciliation exceptions — framed four ways.

Poor

"Fix the reconciliation exceptions in the Meridian account."



09:30 - 11:00 · THE AGENTIC MINDSET SHIFT

The Frame in Practice: Poor, Better, Best

Same task — reviewing open reconciliation exceptions — framed four ways.

Poor

"Fix the reconciliation exceptions in the Meridian account."

No scope, no verification, no deliverable — Claude has to guess what "fixed" means and how to prove it.



09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

The Frame in Practice: Poor, Better, Best

Same task — reviewing open reconciliation exceptions — framed four ways.

Poor

"Fix the reconciliation exceptions in the Meridian account."

No scope, no verification, no deliverable — Claude has to guess what "fixed" means and how to prove it.

Poor

***"Outcome: fix the fee bug.
Scope: the fee module.
Verification: run the tests.
Deliverable: updated code.
Guardrails: don't break anything."***



09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

The Frame in Practice: Poor, Better, Best

Same task — reviewing open reconciliation exceptions — framed four ways.

Poor

"Fix the reconciliation exceptions in the Meridian account."

No scope, no verification, no deliverable — Claude has to guess what "fixed" means and how to prove it.

Poor

***"Outcome: fix the fee bug.
Scope: the fee module.
Verification: run the tests.
Deliverable: updated code.
Guardrails: don't break anything."***

Every ingredient is present but too vague to execute or check against — **completeness isn't quality** (Lab 1).



09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

The Frame in Practice: Poor, Better, Best

Same task — reviewing open reconciliation exceptions — framed four ways.

Poor

"Fix the reconciliation exceptions in the Meridian account."

No scope, no verification, no deliverable — Claude has to guess what "fixed" means and how to prove it.

Poor

**"Outcome: fix the fee bug.
Scope: the fee module.
Verification: run the tests.
Deliverable: updated code.
Guardrails: don't break anything."**

Every ingredient is present but too vague to execute or check against — **completeness isn't quality** (Lab 1).

Better

"Investigate the 12 open exceptions in recon_meridian.csv and classify each as a timing break or a true break."



09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

The Frame in Practice: Poor, Better, Best

Same task — reviewing open reconciliation exceptions — framed four ways.

Poor

"Fix the reconciliation exceptions in the Meridian account."

No scope, no verification, no deliverable — Claude has to guess what "fixed" means and how to prove it.

Poor

**"Outcome: fix the fee bug.
Scope: the fee module.
Verification: run the tests.
Deliverable: updated code.
Guardrails: don't break anything."**

Every ingredient is present but too vague to execute or check against — **completeness isn't quality** (Lab 1).

Better

"Investigate the 12 open exceptions in recon_meridian.csv and classify each as a timing break or a true break."

Scope is named. Still missing verification and a defined deliverable format.



09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

The Frame in Practice: Poor, Better, Best

Same task — reviewing open reconciliation exceptions — framed four ways.

Poor

"Fix the reconciliation exceptions in the Meridian account."

No scope, no verification, no deliverable — Claude has to guess what "fixed" means and how to prove it.

Poor

"Outcome: fix the fee bug. Scope: the fee module. Verification: run the tests. Deliverable: updated code. Guardrails: don't break anything."

Every ingredient is present but too vague to execute or check against — **completeness isn't quality** (Lab 1).

Better

"Investigate the 12 open exceptions in recon_meridian.csv and classify each as a timing break or a true break."

Scope is named. Still missing verification and a defined deliverable format.

Best

"Classify the 12 open exceptions in recon_meridian.csv as timing or true breaks. Verify each against the custodian feed. Deliver a triage table showing the reasoning per exception. Don't auto-clear anything flagged as a true break — escalate those."



09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

The Frame in Practice: Poor, Better, Best

Same task — reviewing open reconciliation exceptions — framed four ways.

Poor

"Fix the reconciliation exceptions in the Meridian account."

No scope, no verification, no deliverable — Claude has to guess what "fixed" means and how to prove it.

Poor

*"Outcome: fix the fee bug.
Scope: the fee module.
Verification: run the tests.
Deliverable: updated code.
Guardrails: don't break anything."*

Every ingredient is present but too vague to execute or check against — **completeness isn't quality** (Lab 1).

Better

"Investigate the 12 open exceptions in recon_meridian.csv and classify each as a timing break or a true break."

Scope is named. Still missing verification and a defined deliverable format.

Best

"Classify the 12 open exceptions in recon_meridian.csv as timing or true breaks. Verify each against the custodian feed. Deliver a triage table showing the reasoning per exception. Don't auto-clear anything flagged as a true break — escalate those."

All five ingredients present: outcome, scope, verification, deliverable, guardrails.



09:30 - 11:00 · THE AGENTIC MINDSET SHIFT

The Loop, Powered by the Frame

Same four stages you just met — now with the frame attached. Claude Code is an agent that runs this loop on every task, not a chatbot, and the frame governs it at every step.



GUARDRAILS ARE DIFFERENT

Guardrails isn't a fifth stage — it's a cross-cutting override that can interrupt *any* stage above (a risk threshold, a protected file). Not a gap in the loop; the thing that can override the loop. *Copilot parallel:* Copilot Chat's Edit/Agent mode gestures at multi-file edits but doesn't run this persistent verify-and-loop cycle by default.



09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

Observe Stage

RESPONSIBLE AI & HUMAN-IN-THE-LOOP

Agent output is never treated as ground truth — state it before anyone touches a keyboard.

The agent reports back

A diff, a test result, a summary — always something you review, never something you rubber-stamp.

You verify independently

Run the tests yourself. Read the diff yourself. The scorer and the test suite are checklists, not judges of truth.

This is the same idea as Guardrails from the last slide, generalized: never trusting the agent's own Observe/report is itself a guardrail. It recurs Day 2 (governance as architecture) and Day 3 (validating extracted rules against ground truth).



09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

WM-101: Advisory Fees Break at Tier Boundaries

Before any tooling talk, here's what you're actually walking into this morning.

The app

`agentic_framing` — a small Wealth Management codebase: portfolios, holdings, advisory fees, client statements.

The issue — WM-101

Advisory fees are calculated incorrectly when a household's AUM crosses a tier breakpoint.

YOUR GOAL FOR LAB 1-1

Ship a safe, tested fix in `agentic_framing/fees.py` — backed by deterministic test evidence, not just a claim that it works.

Read → Decide → Act → Observe, on This Bug

You just saw the loop and the frame generically. Watch them become real on this exact bug.

READ		DECIDE		ACT		OBSERVE	
Agent	Reads fees.py, the failing test, and CLAUDE.md before touching anything.	Agent	Decides whether it has enough to safely act, or needs to ask.	Agent	Edits fees.py inside the scoped boundary.	Agent	Reports back — but the result is never treated as ground truth.
You	Kick off analysis — a slash command can package this exact read every time.	You	Choose plan mode while breakpoint behavior is still unconfirmed, and pick your model.	You	Reference wm-101-fix-flow so the fix follows the same standard every time.	You	Run pytest tests/test_fees.py and review the diff yourself.
		Frame	Outcome ("fees correct at every breakpoint") and Scope (fees.py only) get fixed here.	Frame	Guardrails bound what the skill can touch (never the public signature).	Frame	Verification and Deliverable close the loop.

WATCH IT HAPPEN

Run python -m agentic_framing convert "... " and the app's own inner agent prints [READ] / [DECIDE] / [ACT] / [OBSERVE] as it works — the same four beats, one zoom level down.



09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

A Small-Change Playbook

Turn the applied loop into three concrete choices before you touch a file.

1. CHOOSE YOUR MODE

- Plan**Use it when the acceptance criteria or test shape is still unclear.
- Act**Use it for a bounded edit once the target is clear.

2. CHOOSE YOUR MODEL

- Fast**Use the cheaper model for routine checks and narrow steps.
- Stronger**Use the more capable model when framing or reasoning is the bottleneck.

3. REFERENCE YOUR SKILL

- Frame**Name the skill explicitly so the workflow is visible and repeatable.
- Reuse**Keep the task narrow; let CLAUDE.md and .claude/skills/ carry the standard.

LAB 1-1 HANDOFF

For WM-101, this becomes: use skill `wm-101-fix-flow`, run analysis first, then scoped implementation, then deterministic verification with `python3 -m pytest tests/test_fees.py` and `python3 -m pytest`.



09:30 – 11:00 · THE AGENTIC MINDSET SHIFT

Quick Check: The Agentic Mindset Shift

1. Which loop describes agentic coding?

(a) Read → Act → Observe · (b) Read → Decide → Act → Observe · (c) Prompt → Suggest → Accept

Answer: (b) Read → Decide → Act → Observe

2. Which two artifacts let you package a task frame so it runs the loop again without you rewriting it?

(a) A README and a package.json · (b) CLAUDE.md and a SKILL.md · (c) A prompt and a code comment

Answer: (b) CLAUDE.md and a SKILL.md

3. A SKILL.md is closest, in GitHub Copilot, to:

(a) A Copilot suggestion · (b) A Prompt File · (c) The Copilot sidebar

Answer: (b) A Prompt File

4. What must accompany a shipped change before it counts as "verified," per this morning's principle?

(a) A confident summary from the agent · (b) Test evidence you can actually check · (c) A green checkmark emoji

Answer: (b) Test evidence you can actually check



09:30 - 11:00 · THE AGENTIC MINDSET SHIFT

You Can Now...

- Describe the Read → Decide → Act → Observe loop, and name a task in your day where an agent adds value — and one where it doesn't.
- Package a task frame into a CLAUDE.md rule or a Skill so it runs the same loop without you rewriting it each time.
- Ship one small, verified change, backed by test evidence — not just a claim that it works.

NEXT UP

Lab 1 puts all three to work on a real bug — you'll frame it, ship a verified fix, and see the Skill that packages this exact flow.